

DONINHA AI

Hybrid Neuro-Symbolic Middleware for Epistemological Reasoning in Language Models

Author: Daniel Barros Fonseca

Contact: danielfonseca420@proton.me

Patent pending registration — INPI (Brazil)

Abstract

This paper presents Doninha AI, a hybrid neuro-symbolic software middleware interposed between client applications and external Large Language Models (LLMs). The system operates as a transparent epistemic proxy, intercepting calls bidirectionally, submitting the original prompt to a reasoning pipeline structured across seven functional layers (L1–L7), and returning a response enriched with auditable certainty, contradiction, and truth-value metadata.

The pipeline combines Aristotelian semantic extraction (L1), Kantian classification of propositions (L2), scientific filtering by Hempel's and Popper's criteria (L2.5), paraconsistent evaluation based on Newton da Costa's Annotated Evidence Logic (L3), synthesis by Russellian equivalence with Chain of Verification (L4), text generation by the base LLM (L5), fluency refinement (L6), and delivery of an auditable response with a hierarchical chain-of-thought trace (L7). Unlike existing prompt engineering solutions and post-processing guardrails, Doninha AI acts on the structured pre-processing of the prompt and the epistemological post-processing of the response, without requiring any modification to the client or to the external LLM.

1. Introduction

1.1. The Structural Problem of Current LLMs

Commercially available Large Language Models — including models based on the Transformer architecture such as GPT (OpenAI), Claude (Anthropic), Gemini (Google), and equivalents — are accessed by client applications through calls to standardized APIs, in which the client sends a prompt in natural language and receives a response generated by the model. This paradigm, although widely adopted, presents documented structural limitations that compromise the reliability and audibility of responses in domains with high epistemic demands.

The main limitations are:

- Absence of epistemic pre-processing: the prompt reaches the LLM exactly as formulated by the user, without formal decomposition into concepts, hypotheses, or judgments.
- Lack of uncertainty quantification at input: the LLM receives no information about the degree of certainty, contradiction, or indeterminacy of the propositions in the prompt.
- Absence of pre-generation consistency checking: logical contradictions present in the prompt are transmitted to the model in their entirety.
- Opacity of intermediate reasoning: the transformation process between prompt and response occurs entirely inside the LLM, with no auditable step accessible to the client.
- Dependence on manual prompt engineering: the quality of the response depends on the user's skill in formulating an adequate prompt.
- Inability to handle intermediate truth-values: the classical binary logic of LLMs collapses when faced with propositions that require non-absolute values — a phenomenon known as machine hallucination.

Previous solutions — such as automated prompt engineering systems, output guardrails, and API wrappers — act predominantly on post-processing the response or on superficial filtering of the prompt, without implementing structured epistemological reasoning at the bidirectional interception layer.

1.2. The Doninha AI Proposal

Doninha AI addresses these limitations through an original approach: transparently interposing a middleware layer that applies hybrid neuro-symbolic reasoning to each call before forwarding it to the external LLM. The system combines:

- Statistical natural-language generation by the base LLM (neural component);
- Symbolic structuring of propositions through Aristotelian categories, Kantian judgments, and scientific syllogism;
- Epistemological verification through annotated paraconsistent logic, Russellian synthesis, and Chain of Verification.

Technical definition:

A hybrid neuro-symbolic middleware is a system that combines neural networks — responsible for statistical language generation — with symbolic mechanisms based on logical rules and formal knowledge structures. Unlike purely neural models (which compute token probabilities) or purely symbolic models (which operate exclusively through explicit rules), the hybrid system leverages the strengths of both approaches.

2. General Architecture

2.1. Positioning as a Transparent Epistemic Proxy

The middleware is positioned between the client application and the external LLM, operating across two independent and complementary interception planes:

Plane	Function
Upstream (input)	Intercepts the prompt before its transmission to the external LLM. The original prompt is processed by the L1–L7 pipeline and replaced with an enriched prompt containing hierarchized propositions, quantified degrees of truth, and epistemic calibration instructions.
Downstream (output)	Intercepts the response generated by the external LLM before its delivery to the client. The response is annotated with epistemic metadata — certainty (Gc), contradiction (Gct), paraconsistent state, truth-value (v), and CoVe verification log — and returned in the API's original format.

Transparency is guaranteed by two mechanisms: (i) the middleware implements the same input and output interface as the target external LLM, so that the client application requires no code changes; (ii) the middleware forwards calls to the external LLM using the configured credentials and endpoints, acting as an invisible epistemic proxy.

2.2. Overview of the Complete Flow

The processing flow for each intercepted call follows the sequence below:

Stage	Main Module	Brief Function
L1	ConceptTable + LogicLMSymbolicSolver	Semantic extraction and Aristotelian mapping of prompt terms
L2	KantianJudgmentEngine + BERTAssertionClassifier	Classification and epistemic prioritization via Kant's table of twelve judgments
L2.5 ★	ScientificSyllogismPipeline	Scientific filtering: syllogistic validation, Hempel's criterion, and Popperian falsifiability
L3	ParaconsistentEngine + Weighted Dynamic Ensemble	Assignment of μ , λ , certainty (Gc), contradiction (Gct), truth-value (v), and logical state
L4	RussellianSynthesisEngine + ChainOfVerificationAgent	Synthesis via proposition-world equivalence and verification through CoVe
L5	generate_response + providers	Generation of natural-language textual response with L1–L4 context
L6	FinalResponseEngine	Polishing of clarity, cohesion, and tone; preservation of epistemic data
L7	FinalTextEngine + HierarchicalCoTTrace	Auditable delivery with [AUDIT] blocks and a per-layer chain-of-thought trace

★ Stage present in the current version and absent in previous versions

3. L1–L7 Pipeline: Detailed Description

3.1. L1 — Table of Concepts

The Problem L1 Solves

Purely neural LLMs receive the prompt as a sequence of tokens and probabilistically calculate what the next one should be. This process is blind to fundamental semantic distinctions: synonyms, antonyms, ambiguous terms, and contextually incompatible concepts are all treated the same way. The result is responses that directly reflect the biases and ambiguities of the input.

Kant wrote that "thoughts without content are empty; intuitions without concepts are blind". L1 exists to provide content to the system: it extracts the essential concepts from the prompt, maps the relations between them, and delivers this semantic structure as the foundation for all subsequent layers.

Operation

The main module of L1 is ConceptTable, which operates in four sequential steps:

- Extraction of concepts from the prompt with noise filtering;
- Mapping of binary Aristotelian relations: synonymy, antonymy, hyperonymy (more general term), hyponymy (more specific term), homonymy, and paronymy;
- Querying the local knowledge base (KB) and, when available, the RAG context via injection of vector embeddings;
- Filtering of contextually incompatible concepts by the LogicLM symbolic solver, which checks canonical compatibility before passing the material on to L2.

SpacySemanticTermExtractor is an optional component that enables the identification of complex phrases and multi-word expressions via spaCy — useful for technical domains with specialized terminology. LogicLMSymbolicSolver complements the extraction by checking whether the identified concepts are compatible with the prompt's context, generating canonical alerts when it detects inconsistencies.

Why this matters:

A large share of machine hallucinations stem from LLM errors in identifying the prompt's context. By building an explicit table of concepts before generation, L1 reduces the space of semantic ambiguity that the model must resolve internally during token prediction.

3.2. L2 — Kantian Judgments

The Problem L2 Solves

L1 delivers a set of semantically organized concepts. L2 transforms this material into propositions structured according to formal logical criteria, assigning epistemic priority to each one. Without

this structuring, the paraconsistent engine of L3 would receive raw, unranked concepts — the equivalent of trying to calculate probabilities over unstructured text.

The Table of Twelve Judgments

KantianJudgmentEngine classifies each proposition extracted from the prompt according to the four dimensions of Kant's table of judgments, which structures the human faculty of understanding according to formal categories of thought:

Dimension	Categories
Quantity	Universal ("all"), Particular ("some"), Singular ("this")
Quality	Affirmative, Negative, Infinite
Relation	Categorical ("S is P"), Hypothetical ("if... then"), Disjunctive ("either... or")
Modality	Problematic (possible), Assertoric (actual), Apodictic (necessary)

For each extracted proposition, the engine assigns a degree of epistemic priority (strong, intermediate, weak) and classifies the hypothesis's domain into four operational categories: empirically verified, theoretically plausible, logically contradictory, or empirically indeterminate. This domain functions as a Bayesian prior for the calculation of μ and λ in L3.

BERTAssertionClassifier, when available, complements the Kantian classification with a bidirectional Transformer-based analysis, classifying each proposition as true (T), indeterminate (I), or false (F) according to evidence extracted from the context. The integration between Kantian classification and bidirectional BERT — mediated by the formal structure of the table of judgments — is one of the architecture's technical differentiators.

3.3. L2.5 — Scientific Syllogism (Intermediate Stage)

Why This Stage Exists

L2 can generate Kantianly well-formed hypotheses that present two distinct problems: they are semantically distant from the original prompt (irrelevant, in Hempel's sense) or they are propositions that could not be refuted by any possible evidence (unfalsifiable, in Popper's sense). If forwarded directly to L3, these hypotheses introduce noise into the calculation of the paraconsistent metrics. The intermediate stage filters out this noise before L3.

The Three Filters

- **AristotelianSyllogismValidator:** checks whether the logical relations between premises and conclusion are syllogistically valid. Hypotheses with invalid inferences are flagged and removed.
- **HempelFilter:** applies Carl Hempel's relevance criterion, discarding hypotheses with low semantic relation to the original prompt. It prevents the paraconsistent engine from processing propositions on topics not pertinent to what the user actually asked.
- **PopperFalsifiability:** evaluates whether each hypothesis is empirically testable — that is, whether it could be refuted by some possible evidence. Hypotheses that are too

vague (e.g., absolute statements immune to any counter-example) are removed before reaching the paraconsistent engine.

ScientificSyllogismPipeline orchestrates the three filters in sequence and delivers to L3 only the hypotheses that passed all the criteria. This results in a smaller, more precise set of propositions for evaluation, which improves the quality of the logical states produced by L3.

3.4. L3 — Paraconsistent Engine

The Problem of Logical Explosions

Classical binary logic — adopted by all current commercial LLMs — suffers from a fundamental limitation: when the system encounters a contradiction (proposition P and its negation $\neg P$ both present), any proposition can be proven true. This phenomenon, called trivialization or logical explosion, lies at the root of machine hallucinations: the model, unable to formally handle the contradiction, produces a random or evasive response that reflects no real evidence.

Newton da Costa, a Brazilian mathematician, developed the Annotated Paraconsistent Evidence Logic (LAE) precisely to solve this problem: paraconsistent systems allow local contradictions to exist without the system as a whole collapsing. Da Costa called this a "gentle explosion": local contradictions do not generate global contradictions.

The Paraconsistent Engine: Metrics and States

ParaconsistentEngine evaluates each proposition delivered by L2.5 by calculating five metrics from two input values:

Metric	Formula	Meaning
μ (mu)	Favorable semantic similarity score	Degree of evidence favorable to the proposition $\in [0,1]$
λ (lambda)	Contrary semantic similarity score	Degree of evidence contrary to the proposition $\in [0,1]$
Gc	$Gc = \mu - \lambda$	Degree of certainty $\in [-1, +1]$
Gct	$Gct = \mu + \lambda - 1$	Degree of contradiction $\in [-1, +1]$
v	$v = (\mu - \lambda + 1) / 2$	Normalized truth-value $\in [0,1]$

Based on the values of Gc and Gct , the para-analyzer classifies each proposition into one of twelve logical states of the QUPC lattice (Unit Square of the Cartesian Plane). The four extreme states correspond to the vertices of the square:

- $D = (1, 0)$: True — maximum affirmative certainty, no contradiction;
- $B = (0, 1)$: False — maximum negative certainty, no contradiction;
- $C = (1, 1)$: Inconsistent — high positive contradiction;
- $A = (0, 0)$: Indeterminate — absence of evidence in both directions.

Between these extremes there are eight intermediate states (Near-True, Near-False, Inconsistent_to_T, NT_to_Inconsistent, Indeterminate_to_T, Indeterminate_to_F, NT_to_Indeterminate, and NF_to_T) that allow the system to express epistemic nuances impossible under classical binary logic.

Weighted Dynamic Ensemble: The Neural Dimension of L3

The current version of L3 combines two sources for calculating μ and λ through an ensemble with dynamic weights:

- **Heuristic source:** semantic similarity scoring between the proposition and the knowledge base documents, adjusted by the epistemic priors delivered by L2 (domain and priority of the hypothesis).
- **Neural source (TruthScoringModel):** a Transformer classifier trained specifically to produce a paraconsistent state and truth-value from natural-language propositions.

The ensemble weights are calculated dynamically according to the agreement between the two sources: the heuristic weight equals 0.65 plus 0.25 times the degree of agreement between the sources, while the neural weight corresponds to the complement of the heuristic weight relative to 1.0.

When the two sources agree (high agreement), the heuristic weight rises to as much as 0.90. When they diverge, the ratio approaches 65/35. The system records `ensemble_agreement`, `neural_state`, and `neural_truth` as metadata in the audit blocks. When `TruthScoringModel` is not available, L3 operates in pure heuristic mode with `heuristic_weight` = 1.0.

Two control mechanisms derive from the paraconsistent state:

(1) Additional evidence-search rule: when $Gct \geq +0.5$, the pipeline instructs L1 to broaden the KB query or trigger the web search agent before proceeding; (2) Conclusion-issuance rule: when $Gct < +0.5$ and $|Gc| \geq +0.5$, the system has sufficient certainty and the conclusion is issued to L4 with quality metadata.

3.5. L4 — Russellian Synthesis with Chain of Verification

Theoretical Foundation

L4 is the core of the system's verificational dimension. It is based on the principle established by Bertrand Russell in *The Problems of Philosophy*: the truth of a proposition is not a statistical prediction about which word should come next, but a degree of equivalence between the proposition and the facts represented in a knowledge base. L4 implements this principle computationally.

Operation

L4 operates through three complementary modules:

- **RussellConceptBase:** extracts sentences about correspondence, belief, fact, and truth from Russell's text (data/russell.txt), generates a table of conceptual weights normalized between 0.2 and 1.0, and calculates a Russellian equivalence score for each proposition.
- **RussellianSynthesisEngine:** receives the paraconsistent hypotheses from L3, the Kantian priorities from L2, and the original prompt. It selects the hypothesis with the highest truth-value as the basis for the response and calculates a final weighted truth-value combining Kantian priority (L2), paraconsistent certainty (Gc), and the Russellian equivalence score in a 0.7/0.3 ratio.
- **ChainOfVerificationAgent:** applies the Chain of Verification (CoVe) pattern in the Factor + Revise variant. It generates between six and twelve independent verification questions about the synthesized response, classifies each one as Confirmed, Refuted, Partially correct, or Uncertain, and rewrites the response incorporating only what was actually verified.

The result is encapsulated in the `SynthesisResult` structure, which gathers the revised response, the aggregated truth-value, the certainty (Gc), the degree of contradiction (Gct), the logical state, the supporting evidence, the falsified hypotheses, the verification log, and a confidence label automatically derived (ranging from "Indeterminate" to "High Confidence").

3.6. L5, L6, and L7 — Generation, Refinement, and Final Synthesis

L5 — Text Generation

L5 transforms L4's structured synthesis into fluent natural language. The differentiator compared to a common LLM is that the model does not start from scratch: it receives structured context comprising concepts (L1), prioritized propositions (L2), paraconsistent states (L3), and verified synthesis (L4), which significantly reduces the space of possible hallucinations during generation.

The `generate_response` module routes generation to the configured provider (Ollama, OpenAI, Anthropic, Gemini, Grok, Groq, Meta/Llama, local custom_lm, or template fallback), ensuring operation even without an available external provider.

L6 — Final Refinement

L6 polishes the text generated in L5: it improves clarity, fluency, and cohesion; adjusts the tone to the calculated confidence level; explicitly mentions uncertainties and contradictions when present; and ensures terminological consistency with L1's table of concepts. `EpistemicContext` aggregates the states from L3, the paraconsistent routes, and the BERT classifications so that the refinement is informed by the complete epistemic context.

L7 — Final Auditable Synthesis

L7 integrates the results from L1–L6 into the final response delivered to the client. The layer classifies the audience profile (layperson, technical, or academic) and adjusts the tone and level of detail accordingly. More importantly: L7 adds audit blocks ([AUDIT L4], [AUDIT L5], [AUDIT L6], [AUDIT L7]) that record the provider used, the model, truth-value, certainty, contradiction,

logical state, bibliographic sources, and, when requested, the complete hierarchical Chain of Thought trace per layer (HierarchicalCoTTrace).

Responsible use in sensitive domains:

The presence of [AUDIT] blocks transforms the response from a black box into a traceable document. In fields such as law, medicine, education, and science, where the source and degree of certainty of a statement are legally and methodologically relevant, this auditability is a concrete operational differentiator, not merely an architectural feature.

4. Supporting Components

4.1. Hybrid RAG and Knowledge Base

The system supports four context-retrieval strategies, configurable by domain:

- **direct_injection**: direct injection of KB documents into the prompt's context — faster and deterministic.
- **semantic_retrieval**: vector-similarity search in ChromaDB using HuggingFace Embeddings — retrieves the documents most semantically close to the prompt.
- **hybrid**: combines the two previous strategies, balancing speed and semantic coverage.
- **domain_aware**: automatically detects the prompt's domain (medicine, law, science, etc.) and selects the domain-specific KB before retrieving documents.

The Knowledge Base can be loaded from JSON, JSONL, or text documents, enriched via ChromaDB, merged across sources, and accessed by specific domain. The minimal fallback base (SEED_KNOWLEDGE_BASE) ensures operation even without a configured KB.

4.2. Web Search Agent

The agent implemented in `agente_busca_web.py` combines local search in ChromaDB with external search via DuckDuckGo in a ReAct (Reasoning + Acting) agent built on LangChain. Its activation is not generic — it is conditioned on the epistemic state produced by L3:

- Indeterminate state ($Gct \leq -0.5$): absence of evidence in both directions → agent activated to search for external context;
- Active heuristic fallback (TruthScoringModel unavailable) → agent activated as compensatory enrichment;
- Inconsistent state ($Gct \geq +0.5$): high contradiction → agent activated to search for evidence that allows the contradiction to be resolved.

This conditional activation transforms web search from a generic enrichment of the prompt into a resolution mechanism specifically targeted at problematic epistemic states.

4.3. Customizable Language Model

For deployments that require fully local operation — without dependence on any external provider — the system includes complete infrastructure for training its own language model: a SentencePiece tokenizer (`custom_tokenizer.py`), a custom Transformer architecture (`custom_lm_model.py`), and pre-training routines on a local philosophical corpus (`pretrain_custom_lm.py`, `run_pretrain.py`). When configured as the `custom_lm` provider, the system operates entirely on-premise.

4.4. Auditable Chain of Thought

HierarchicalCoTTrace records one step per pipeline layer (CoTStep), containing the layer, title, reasoning summary, key decisions, output, and duration. The complete trace can be exported as

a structured dictionary or as readable Markdown, and is accessible via `return_cot=True` in the pipeline. This allows client integrations to inspect the system's intermediate reasoning without access to the source code.

5. Interfaces and Providers

5.1. Available Interfaces

Interface	File	Usage
CLI	pipeline.py	Command line: --prompt, --demo, --repl
REST API	api.py	FastAPI with /health, /process, /chat, /agent endpoints
Chat UI	app.py	Chainlit interface with streaming via Ollama
Standalone	doninha_standalone.py	Consolidated file for distribution without separate dependencies

5.2. Supported LLM Providers

The `llm_provider_client.py` module normalizes access to nine external LLM providers, plus two local modes:

Provider	Type
Ollama	Local — doninha8:latest model (recommended default)
OpenAI	Remote — compatible with the OpenAI API
Anthropic (Claude)	Remote — via ANTHROPIC_API_KEY
Google Gemini	Remote — via GEMINI_API_KEY
xAI Grok	Remote — via GROK_API_KEY
Groq	Remote — accelerated inference, compatible with OpenAI
Meta / Llama API	Remote — access to Llama models via the Meta API
custom_lm	Local — model trained on the organization's own infrastructure
template	Local fallback with no external call — ensures fully offline operation

6. Theoretical Foundations

Doninha AI is the implemented proof of concept of the propositions developed by its creator in the paper "A True Epistemology for Artificial Intelligence". The main philosophical and scientific references underlying the architecture are:

Reference	Layer	Contribution to the System
Aristotle (Topics / Categories)	L1, L2.5	Fundamental semantic relations (synonymy, antonymy, hypo/hyperonymy) and syllogistic validation
Immanuel Kant (Critique of Judgment)	L2	Table of twelve judgments as a formal structure for classifying propositions; analytic/synthetic, a priori/a posteriori distinction
Newton da Costa (LAE)	L3	Annotated Paraconsistent Evidence Logic; formal treatment of contradictions without trivializing the system
Bertrand Russell (Problems of Philosophy)	L4	Correspondence theory and equivalence as a principle for aligning propositions with the real world
Carl Hempel (Relevance Criterion)	L2.5	Filtering of hypotheses semantically irrelevant to the prompt
Karl Popper (Falsifiability)	L2.5	Criterion of scientific demarcation: elimination of unfalsifiable propositions
Antoine Culioli (Enunciative Linguistics)	L2	Enunciative analysis integrated with BERT classification for the semantic contextualization of judgments

7. Technical Differentiators and Innovation

The following are the aspects that distinguish Doninha AI from existing LLM integration solutions:

- **First transparent middleware for bidirectional interception** that implements structured epistemological reasoning across multiple layers over calls to external LLMs, without modifying the client or the LLM.
- **First combined application, in an operational software system**, of: (a) Aristotelian table of concepts; (b) Kantian table of judgments; (c) paraconsistent logic with $\mu/\lambda/Gc/Gct/v$ metrics; (d) synthesis by Russellian equivalence; and (e) Chain of Verification — as a sequential pipeline for prompt enrichment for LLMs.
- **Original technical resolution of the problem of logical explosions** in language systems, by replacing the classical binary-logic paradigm with paraconsistent evaluation that preserves intermediate truth-values.
- **Generation of an epistemic audit trail integrated into the response flow**, allowing the verifiability of intermediate reasoning in a way accessible to the client application — a functionality that does not exist in previous systems.
- **Modular architecture** that allows independent replacement or update of any layer (L1–L7) without impacting the others, enabling incremental evolution and adaptation to different base LLMs.
- **Native support for hybrid RAG** integrated into the epistemological pipeline, with configurable strategies for combining local vector retrieval and external search.
- **Pre-paraconsistent scientific filtering** (Hempel + Popper) that eliminates irrelevant and unfalsifiable hypotheses before the calculation of the epistemic metrics — reducing input noise in L3.
- **Weighted Dynamic Ensemble in L3** that combines symbolic heuristics with a trainable neural model (TruthScoringModel) through weights dynamically calculated from the agreement between the two sources.

The theoretical foundation of this approach and the proposal for overcoming it through a hybrid model were developed by the creator and presented in the paper "A True Epistemology for Artificial Intelligence" (Daniel Barros Fonseca), prior to the system's implementation. Doninha AI constitutes the implemented proof of concept of these original theoretical propositions.

8. Evaluation and Metrics

The current version includes systematic evaluation infrastructure, composed of objective metrics and per-component test suites:

Metric / Test	What it evaluates
coherence_l3	Coherence between the truth-value, logical state, and contradiction produced by L3
BLEU	N-gram overlap between the generated response and the reference
ROUGE-L	Longest common subsequence between the generated response and the reference
Semantic similarity	Vector distance via SentenceTransformers between the response and the reference
test_epistemic_classification	Correctness of the epistemic classification in L2 and L3
test_l7_tone_guidance	Appropriateness of tone and level of detail for each audience profile in L7
test_citation_behavior	Presence and correct format of sources in the [AUDIT] blocks
test_rag_hybrid	Quality and relevance of the context retrieved by the RAG strategies

9. Configuration and Execution

9.1. Minimal Configuration

The `config.yaml` file centralizes all the pipeline parameters. Minimal configuration for local operation:

In the generation section, the provider is set to Ollama, with the `doninha8:latest` model and the local address `http://localhost:11434`. In the finalization section, the same provider and model are repeated. In the section referring to L7, the Ollama provider is likewise configured with the `doninha8:latest` model.

9.2. Basic Execution

After installing the dependencies and starting Ollama:

- Process a single prompt via the command line, providing the desired text directly as an execution parameter;
- Start interactive mode (REPL), which maintains a continuous session of questions and answers;
- Start the REST API, making the system's endpoints available for integration with other applications.

9.3. Example Output with Audit

A typical system response includes the natural-language text generated by layers L5 through L7, followed by the audit blocks with the following metadata:

Audit Block	Recorded Metadata
[AUDIT L4]	Provider and model used (ollama / doninha8:latest); logical state (e.g., True); truth-value (e.g., 0.87); degree of certainty Gc (e.g., +0.74); degree of contradiction Gct (e.g., 0.02); sources consulted (e.g., data/russell.txt, local knowledge base)
[AUDIT L7]	Identified audience profile (e.g., technical); ensemble agreement level (e.g., 0.82); neural state (e.g., True); confidence label (e.g., High Confidence)

10. Conclusion

Doninha AI represents a fundamentally different approach to integrating LLMs into software systems. Instead of treating the language model as a direct-access oracle — in which the prompt goes in and the response comes out without structured mediation — the middleware interposes an epistemological reasoning pipeline that makes the process transparent, auditable, and formally grounded.

The combination of Aristotelian conceptual extraction, Kantian classification, scientific filtering by Hempel and Popper, paraconsistent evaluation based on Newton da Costa, and verification by Russellian equivalence is not an aesthetic choice — each element solves a concrete technical problem identified in purely neural LLMs: unresolved semantic ambiguity, absence of epistemic hierarchy, irrelevant or unfalsifiable hypotheses, logical explosions in the face of local contradictions, and opacity of intermediate reasoning.

The system finds natural application in domains where the reliability and traceability of AI responses are legally or methodologically relevant: medicine, law, science, education, and any context in which "the model said so" is not sufficient without knowing why, with what degree of certainty, and based on what evidence.

Licensing:

Use of Doninha AI is permitted only for private users. For commercial or governmental use, contact the developer at danielfonseca420@proton.me for information about licensing in exchange for royalty payments. Doninha AI technology is currently undergoing patent registration with INPI (Brazil).

Daniel Barros Fonseca

Inventor and Architect of Doninha AI

Theoretical foundation: "A True Epistemology for Artificial Intelligence"